

VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



Microprocessors Microcontrollers

Lab 2 report

Timer Interrupt and LED Scanning

Advisor(s): Phan Van Sy

Student(s): Nguyen Thanh Luan 2452711

HO CHI MINH CITY, OCTOBER 2026



Contents

1	Exercise 1	4
2	Exercise 2	7
3	Exercise 3	12
4	Exercise 4	15
5	Exercise 5	17
6	Exercise 6	18
7	Exercise 7	19
8	Exercise 8	20
9	Exercise 9	24
10	Exercise 10	27

List of Figures

1.1	<i>Simulation schematic in Proteus</i>	4
1.2	Proteus Schematic for Exercise 1. The source code and project files can be accessed at: GitHub Repository	5
2.1	<i>Simulation schematic in Proteus</i>	8
2.2	Proteus Schematic for Exercise 2. The source code and project files can be accessed at: GitHub Repository	8
9.1	<i>LED matrix is added to the simulation</i>	25
9.2	Proteus Schematic for Exercise 9. The source code and project files can be accessed at: GitHub Repository	25

List of Tables



Listings

1 Exercise 1

The first exercise show how to interface for multiple seven segment LEDs to STM32F103C6 micro-controller (MCU). Seven segment displays are common anode type, meaning that the anode of all LEDs are tied together as a single terminal and cathodes are left alone as individual pins.

In order to save the resource of the MCU, individual cathode pins from all the seven segment LEDs are connected together, and connect to 7 pins of the MCU. These pins are popular known as the **signal pins**. Meanwhile, the anode pin of each seven segment LEDs are controlled under a power enabling circuit, for instance, an PNP transistor. At a given time, only one seven segment LED is turned on. However, if the delay is small enough, it seems that all LEDs are enabling.

Implement the circuit simulation in Proteus with two 7-SEGMENT LEDs as following:

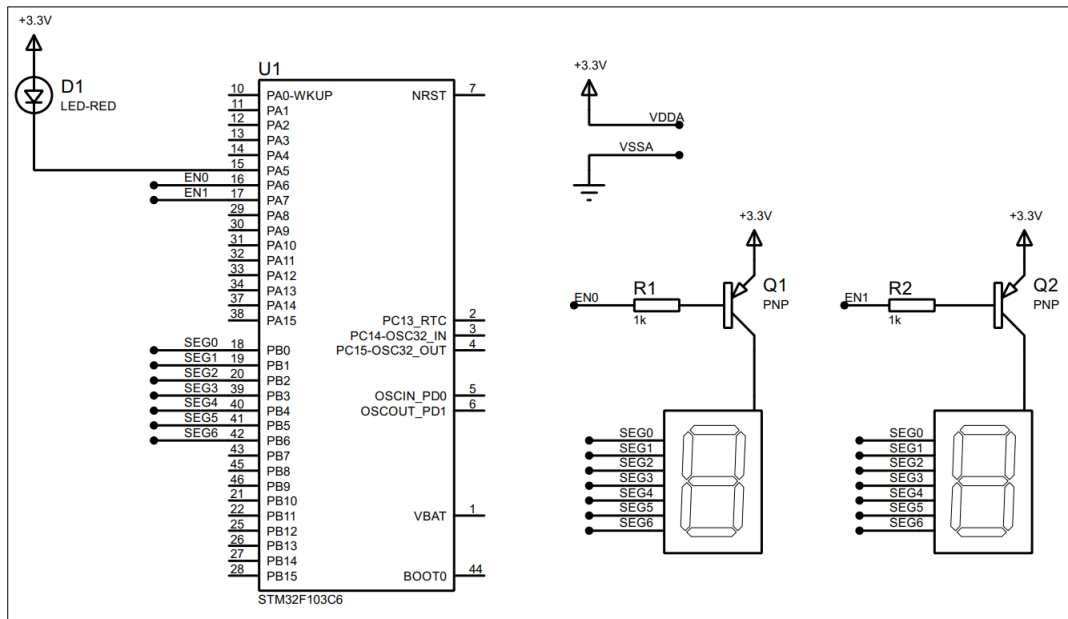


Figure 1.1: *Simulation schematic in Proteus*

Components used in the schematic are listed below:

- 7SEG-COM-ANODE (connected from PB0 to PB6)
- LED-RED

- PNP
- RES
- STM32F103C6

Students are proposed to use the function **display7SEG(int num)** in the Lab 1 in this exercise. Implement the source code in the interrupt callback function to display number "1" on the first seven segment and number "2" for second one. The switching time between 2 LEDs is half of second.

Report 1: Capture your schematic from Proteus and show in the report.

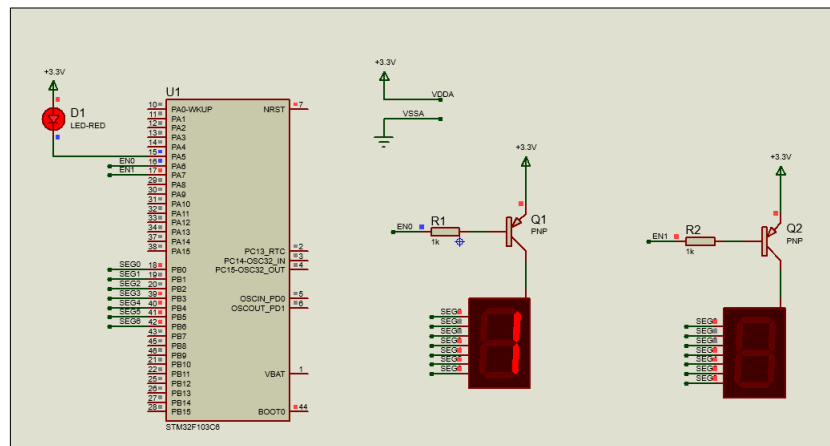


Figure 1.2: Proteus Schematic for Exercise 1. The source code and project files can be accessed at: [GitHub Repository](#)

Report 2: Present your source code in the **HAL_TIM_PeriodElapsedCallback** function.

```
1 void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim) {  
2     timerRun();  
3 }
```

```
1 //This is header file to declare timerRun() function to use  
2 // extra clock in the STM32F103C6  
3  
4 #ifndef INC_SOFTWARE_TIMER_H_  
5 #define INC_SOFTWARE_TIMER_H_
```

```
6
7  extern int timer1_flag;
8
9  void setTimer1(int duration);
10 void timerRun();
11
12 #endif /* INC_SOFTWARE_TIMER_H_ */
```

```
1  //This is header file to implemment timerRun() function to
2  //use extra clock in the STM32F103C6
3
4  #include "software_timer.h"
5
6  int timer1_counter = 0;
7  int timer1_flag = 0;
8
9  void setTimer1(int duration) {
10     timer1_counter = duration;
11     timer1_flag     = 0;
12 }
13
14 void timerRun() {
15     if (timer1_counter > 0) {
16         timer1_counter--;
17         if(timer1_counter <= 0) {
18             timer1_flag = 1;
19         }
20     }
21 }
```

```
1  //this is while loop to run to demonstate the schematic
2
3  setTimer1(50);
4  while (1)
```

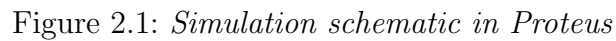


```
5 {
6     if (timer1_flag == 1) {
7         setTimer1(50); //Reset counter for next 500ms
8         if (led_state == 0) {
9             HAL_GPIO_WritePin(EN1_GPIO_Port, EN1_Pin, GPIO_PIN_SET);
10            display7SEG(1);
11            HAL_GPIO_WritePin(EN0_GPIO_Port, EN0_Pin, GPIO_PIN_RESET);
12
13            led_state = 1;
14        } else {
15            HAL_GPIO_WritePin(EN0_GPIO_Port, EN0_Pin, GPIO_PIN_SET);
16            display7SEG(2);
17            HAL_GPIO_WritePin(EN1_GPIO_Port, EN1_Pin, GPIO_PIN_RESET);
18
19            led_state = 0;
20        }
21    }
22    /* USER CODE END WHILE */
23    /* USER CODE BEGIN 3 */
24 }
25 /* USER CODE END 3 */
```

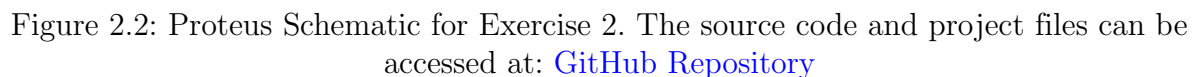
Short question: What is the frequency of the scanning process?

2 Exercise 2

Extend to 4 seven segment LEDs and two LEDs (connected to PA4, labeled as **DOT**) in the middle as following:



Report 1: Capture your schematic from Proteus and show in the report.



8



```
1 void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim) {
2     if (htim->Instance == TIM2) {
3         timerRun();
4     }
5 }
```

```
1 #ifndef INC_SOFTWARE_TIMER_H_
2 #define INC_SOFTWARE_TIMER_H_
3
4 extern int timer1_flag;
5 extern int timer2_flag;
6
7 void setTimer1(int duration);
8 void setTimer2(int duration);
9 void timerRun();
10
11 #endif /* INC_SOFTWARE_TIMER_H_ */
```

```
1 #include "software_timer.h"
2
3 int timer1_counter = 0;
4 int timer1_flag = 0;
5 int timer2_counter = 0;
6 int timer2_flag = 0;
7
8 void setTimer1(int duration) {
9     timer1_counter = duration;
10    timer1_flag = 0;
11 }
12
13 void setTimer2(int duration) {
14     timer2_counter = duration;
15     timer2_flag = 0;
16 }
```



```
17
18 void timerRun() {
19     if (timer1_counter > 0) {
20         timer1_counter--;
21         if (timer1_counter <= 0) timer1_flag = 1;
22     }
23     if (timer2_counter > 0) {
24         timer2_counter--;
25         if (timer2_counter <= 0) timer2_flag = 1;
26     }
27 }
```

```
1  /* USER CODE BEGIN 0 */
2  int led_state = 0;
3  const int MAX_LED = 4;
4  int index_led = 0;
5  int led_buffer[4] = {1, 2, 3, 4};
6
7  int hour = 12, minute = 30, second = 50;
8  /* USER CODE END 0 */
9
10
11
12 int main(void)
13 {
14     /* USER CODE BEGIN 2 */
15     HAL_TIM_Base_Start_IT(&htim2);
16
17     HAL_GPIO_WritePin(GPIOA, EN0_Pin|EN1_Pin|EN2_Pin|EN3_Pin|DOT_Pin,
18                       GPIO_PIN_SET);
19     setTimer1(50);
20     setTimer2(100);
21     /* USER CODE END 2 */
22
23     /* Infinite loop */
```

```
23  /* USER CODE BEGIN WHILE */
24  updateClockBuffer();
25  while (1)
26  {
27      if (timer1_flag == 1) {
28          setTimer1(50);
29          HAL_GPIO_WritePin(GPIOA, EN0_Pin|EN1_Pin|EN2_Pin|EN3_Pin,
30                             GPIO_PIN_SET);
31
32          if (led_state == 0) {
33              display7SEG(1);
34              HAL_GPIO_WritePin(EN0_GPIO_Port, EN0_Pin,
35                               GPIO_PIN_RESET);
36              led_state = 1;
37          }
38          else if (led_state == 1) {
39              display7SEG(3);
40              HAL_GPIO_WritePin(EN1_GPIO_Port, EN1_Pin,
41                               GPIO_PIN_RESET);
42              led_state = 2;
43          }
44          else if (led_state == 2) {
45              display7SEG(3);
46              HAL_GPIO_WritePin(EN2_GPIO_Port, EN2_Pin,
47                               GPIO_PIN_RESET);
48              led_state = 3;
49          }
50          else if (led_state == 3) {
51              display7SEG(0);
52              HAL_GPIO_WritePin(EN3_GPIO_Port, EN3_Pin,
53                               GPIO_PIN_RESET);
54              led_state = 0;
55          }
56      }
57  }
```



```
53     if (timer2_flag == 1) {
54         setTimer2(100);
55         HAL_GPIO_TogglePin(DOT_GPIO_Port, DOT_Pin);
56     }
57
58     /* USER CODE END WHILE */
59     /* USER CODE BEGIN 3 */
60 }
61 /* USER CODE END 3 */
62 }
```

3 Exercise 3

Implement a function named **update7SEG(int index)**. An array of 4 integer numbers are declared in this case. The code skeleton in this exercise is presented as following:

```
1  const int MAX_LED = 4;
2  int index_led = 0;
3  int led_buffer[4] = {1, 2, 3, 4};
4  void update7SEG(int index){
5      switch (index){
6          case 0:
7              //Display the first 7SEG with led_buffer[0]
8              break;
9          case 1:
10             //Display the second 7SEG with led_buffer[1]
11             break;
12          case 2:
13             //Display the third 7SEG with led_buffer[2]
14             break;
15          case 3:
16             //Display the forth 7SEG with led_buffer[3]
17             break;
18          default:
19             break;
```



```
20     }  
21 }
```

This function should be invoked in the timer interrupt, e.g `update7SEG(index_led++)`. The variable `index_led` is updated to stay in a valid range, which is from 0 to 3.

Report 1: Present the source code of the `update7SEG` function.

```
1  void update7SEG(int index) {  
2      HAL_GPIO_WritePin(GPIOA, EN0_Pin|EN1_Pin|EN2_Pin|EN3_Pin,  
3          GPIO_PIN_SET);  
4      switch (index) {  
5          case 0:  
6              display7SEG(led_buffer[0]);  
7              HAL_GPIO_WritePin(EN0_GPIO_Port, EN0_Pin, GPIO_PIN_RESET);  
8              break;  
9          case 1:  
10             display7SEG(led_buffer[1]);  
11             HAL_GPIO_WritePin(EN1_GPIO_Port, EN1_Pin, GPIO_PIN_RESET);  
12             break;  
13          case 2:  
14             display7SEG(led_buffer[2]);  
15             HAL_GPIO_WritePin(EN2_GPIO_Port, EN2_Pin, GPIO_PIN_RESET);  
16             break;  
17          case 3:  
18             display7SEG(led_buffer[3]);  
19             HAL_GPIO_WritePin(EN3_GPIO_Port, EN3_Pin, GPIO_PIN_RESET);  
20             break;  
21          default:  
22             break;  
23      }  
24 }  
25  
26 void display7SEG(int num) {  
    if (num < 0 || num > 9) return;
```

```
27
28  uint8_t code = segment_code[num];
29
30  HAL_GPIO_WritePin(SEG0_GPIO_Port, SEG0_Pin, (code & 0x01) ?
    GPIO_PIN_SET : GPIO_PIN_RESET);
31  HAL_GPIO_WritePin(SEG1_GPIO_Port, SEG1_Pin, ((code >> 1) & 0x01) ?
    GPIO_PIN_SET : GPIO_PIN_RESET);
32  HAL_GPIO_WritePin(SEG2_GPIO_Port, SEG2_Pin, ((code >> 2) & 0x01) ?
    GPIO_PIN_SET : GPIO_PIN_RESET);
33  HAL_GPIO_WritePin(SEG3_GPIO_Port, SEG3_Pin, ((code >> 3) & 0x01) ?
    GPIO_PIN_SET : GPIO_PIN_RESET);
34  HAL_GPIO_WritePin(SEG4_GPIO_Port, SEG4_Pin, ((code >> 4) & 0x01) ?
    GPIO_PIN_SET : GPIO_PIN_RESET);
35  HAL_GPIO_WritePin(SEG5_GPIO_Port, SEG5_Pin, ((code >> 5) & 0x01) ?
    GPIO_PIN_SET : GPIO_PIN_RESET);
36  HAL_GPIO_WritePin(SEG6_GPIO_Port, SEG6_Pin, ((code >> 6) & 0x01) ?
    GPIO_PIN_SET : GPIO_PIN_RESET);
37 }
```

Report 2: Present the source code in the `HAL_TIM_PeriodElapsedCallback`.

```
1  void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim) {
2      if (htim->Instance == TIM2) {
3          timerRun();
4      }
5  }
```

Operational Logic Explanation:

- The `HAL_TIM_PeriodElapsedCallback` function is the Interrupt Service Routine (ISR) that is automatically called by the microcontroller whenever the hardware Timer 2 overflows (configured for a 10ms period).
- In Exercise 3, to prevent nested interrupts and system bottlenecks, all complex display control logic (such as the `update7SEG` or `display7SEG` functions) has been moved out to the `while(1)` loop in the `main()` function.



- This interrupt function is designed to be absolutely minimal: its sole responsibility is to call the `timerRun()` function to decrement the software timer counters (`timer_counter`). When a counter reaches 0, its corresponding flag (`timer_flag`) is set to 1 to signal the main program at the application layer to execute the specific task. This architecture ensures smooth multitasking, extremely fast interrupt response times, and overall system stability.

4 Exercise 4

Change the period of invoking `update7SEG` function in order to set the frequency of 4 seven segment LEDs to 1Hz. The DOT is still blinking every second.

Report 1:

```
1  /* USER CODE BEGIN 2 */
2  HAL_TIM_Base_Start_IT(&htim2);
3
4  HAL_GPIO_WritePin(GPIOA, EN0_Pin|EN1_Pin|EN2_Pin|EN3_Pin|DOT_Pin,
5  GPIO_PIN_SET);
6
7  setTimer1(25);
8  setTimer2(100);
9  /* USER CODE END 2 */
10
11 updateClockBuffer();
12 while (1)
13 {
14     if (timer1_flag == 1) {
15         setTimer1(25);
16
17         update7SEG(index_led);
18
19         index_led++;
20         if (index_led >= MAX_LED) {
21             index_led = 0;
```

```
21     }
22 }
23
24 if (timer2_flag == 1) {
25     setTimer2(100);
26     HAL_GPIO_TogglePin(DOT_GPIO_Port, DOT_Pin);
27
28     // logic counting time
29     second++;
30     if (second >= 60) {
31         second = 0;
32         minute++;
33     }
34     if (minute >= 60) {
35         minute = 0;
36         hour++;
37     }
38     if (hour >= 24) {
39         hour = 0;
40     }
41     updateClockBuffer();
42 }
43 }
```

Operational Logic Explanation: In Exercise 4, the parameter passed to `timer1` is reduced from 50 to 25. Since the base interrupt period of the hardware (Timer 2) is 10ms, the actual counting time of `timer1` will be $25 \times 10\text{ms} = 250\text{ms}$. The global variable `index_led` (ranging from 0 to 3) is used as a parameter for the `update7SEG()` function. Every 250ms, the application program outputs the data of 1 LED to the GPIO pins and simultaneously turns off the enable pins of the other 3 LEDs to prevent ghosting. The time required to complete a full scanning loop for all 4 LEDs is $250\text{ms} \times 4 = 1000\text{ms}$ (1 second). Thanks to this time-division multiplexing design, the human eye will perceive all 4 LEDs as being lit simultaneously, and this speed is perfectly synchronized with the actual ticking seconds of the system.

5 Exercise 5

Implement a digital clock with **hour** and **minute** information displayed by 2 seven segment LEDs. The code skeleton in the **main** function is presented as follows:

```
1  int hour = 15, minute = 8, second = 50;
2
3  while(1){
4      second++;
5      if (second >= 60){
6          second = 0;
7          minute++;
8      }
9      if(minute >= 60){
10         minute = 0;
11         hour++;
12     }
13     if(hour >=24){
14         hour = 0;
15     }
16     updateClockBuffer();
17     HAL_Delay(1000);
18 }
```

The function **updateClockBuffer** will generate values for the array **led_buffer** according to the values of hour and minute. In the case these values are 1 digit number, digit 0 is added.

Report 1: Present the source code in the **updateClockBuffer** function.

```
1  void updateClockBuffer() {
2      // hours
3      led_buffer[0] = hour / 10;
4      led_buffer[1] = hour % 10;
5
6      // minutes
```



```
7   led_buffer[2] = minute / 10;  
8   led_buffer[3] = minute % 10;  
9 }
```

6 Exercise 6

```
1   setTimer0 (1000) ;  
2   while (1) {  
3       if ( timer0_flag == 1) {  
4           HAL_GPIO_TogglePin( LED_RED_GPIO_Port, ED_RED_Pin);  
5           setTimer0 (2000) ;  
6       }  
7   }
```

Report 1: If line 1 of the code above is missing, what happens after that and why?

- **Result:** The LED will never toggle.
- **Why:** The variables `timer0_counter` and `timer0_flag` are initialized to 0 at the global declaration. If `setTimer0(1000);` is not called before entering the `while(1)` loop, `timer0_counter` remains 0. Because the `timer_run()` function only processes counters strictly greater than 0 (`if(timer0_counter > 0)`), it will never decrement the counter and will never set `timer0_flag` to 1. Consequently, the `if(timer0_flag == 1)` condition in the main loop is never satisfied.

Report 2: If line 1 of the code above is changed to `setTimer0(1)`, what happens after that and why?

- **Result:** The LED will also never toggle, exactly like the scenario in Report 1.
- **Why:** The function `setTimer0(duration)` calculates the counter using integer division: `duration / TIMER_CYCLE`. Since `TIMER_CYCLE` is 10, passing 1 results in `1 / 10`, which evaluates to 0 in C integer math. The `timer0_counter` is set to 0, meaning the `timer_run()` function will ignore it, the flag will remain 0, and the LED toggling block will never execute.

Report 3: If line 1 of the code above is changed to `setTimer0(10)`, what is changed compared to the 2 first questions and why?



- **Result:** The LED will toggle almost immediately (after 10ms) for the first time, and then it will continuously toggle every 2000ms.
- **Why:** This behavior is drastically different from the first two cases because $10 / 10$ evaluates to 1. The `timer0_counter` is successfully initialized to 1. At the very first hardware timer interrupt (10ms later), `timer_run()` detects that $1 > 0$, decrements the counter to 0, and immediately sets `timer0_flag = 1`. The `while(1)` loop catches this flag, toggles the LED, and safely sets the subsequent timer duration to 2000ms (`setTimer0(2000)`). This proves that the input duration must be greater than or equal to the `TIMER_CYCLE` for the software timer to trigger its first cycle successfully.

7 Exercise 7

Upgrade the source code in Exercise 5 (update values for hour, minute and second) by using the software timer and remove the `HAL_Delay` function at the end. Moreover, the DOT (connected to PA4) of the digital clock is also moved to main function.

Report 1: Present your source code in the while loop on main function.

```
1  /* Infinite loop */
2  /* USER CODE BEGIN WHILE */
3
4  updateClockBuffer();
5  while (1)
6  {
7      if (timer2_flag == 1) {
8          setTimer2(100);
9          HAL_GPIO_TogglePin(DOT_GPIO_Port, DOT_Pin);
10
11         // logic counting time
12         second++;
13         if (second >= 60) {
14             second = 0;
15             minute++;
16         }
```



```
17     if (minute >= 60) {
18         minute = 0;
19         hour++;
20     }
21     if (hour >= 24) {
22         hour = 0;
23     }
24
25     updateClockBuffer();
26 }
27
28 /* USER CODE END WHILE */
29
30 /* USER CODE BEGIN 3 */
31 }
32 /* USER CODE END 3 */
```

8 Exercise 8

Move also the `update7SEG()` function from the interrupt timer to the main. Finally, the timer interrupt only used to handle software timers. All processing (or complex computations) is move to an infinite loop on the main function, optimizing the complexity of the interrupt handler function.

Report 1: Present your source code in the the main function. In the case more extra functions are used (e.g. the second software timer), present them in the report as well.

```
1  int main(void)
2  {
3      /* USER CODE BEGIN 1 */
4
5      /* USER CODE END 1 */
6
7      /* MCU
Configuration-----*/
```



```
8
9      /* Reset of all peripherals, Initializes the Flash interface and
10     the SysTick. */
11     HAL_Init();
12
13     /* USER CODE BEGIN Init */
14
15     /* USER CODE END Init */
16
17     /* Configure the system clock */
18     SystemClock_Config();
19
20     /* USER CODE BEGIN SysInit */
21
22     /* USER CODE END SysInit */
23
24     /* Initialize all configured peripherals */
25     MX_GPIO_Init();
26     MX_TIM2_Init();
27
28     /* USER CODE BEGIN 2 */
29
30     HAL_TIM_Base_Start_IT(&htim2);
31
32     HAL_GPIO_WritePin(GPIOA, EN0_Pin|EN1_Pin|EN2_Pin|EN3_Pin|DOT_Pin,
33     GPIO_PIN_SET);
34     setTimer1(25);
35     setTimer2(100);
36
37     /* USER CODE END 2 */
38
39     /* Infinite loop */
40     /* USER CODE BEGIN WHILE */
41
42     updateClockBuffer();
43     while (1)
```



```
41 {
42     if (timer1_flag == 1) {
43         setTimer1(25);
44
45         update7SEG(index_led);
46
47         index_led++;
48         if (index_led >= MAX_LED) {
49             index_led = 0;
50         }
51     }
52
53     if (timer2_flag == 1) {
54         setTimer2(100);
55         HAL_GPIO_TogglePin(DOT_GPIO_Port, DOT_Pin);
56
57         // logic counting time
58         second++;
59         if (second >= 60) {
60             second = 0;
61             minute++;
62         }
63         if (minute >= 60) {
64             minute = 0;
65             hour++;
66         }
67         if (hour >= 24) {
68             hour = 0;
69         }
70
71         updateClockBuffer();
72     }
73
74     /* USER CODE END WHILE */
75
```



```
76  /* USER CODE BEGIN 3 */
77  }
78  /* USER CODE END 3 */
79  }
```

```
1  void update7SEG(int index) {
2      HAL_GPIO_WritePin(GPIOA, EN0_Pin|EN1_Pin|EN2_Pin|EN3_Pin,
3      GPIO_PIN_SET);
4      switch (index) {
5          case 0:
6              display7SEG(led_buffer[0]);
7              HAL_GPIO_WritePin(EN0_GPIO_Port, EN0_Pin, GPIO_PIN_RESET);
8              break;
9          case 1:
10             display7SEG(led_buffer[1]);
11             HAL_GPIO_WritePin(EN1_GPIO_Port, EN1_Pin, GPIO_PIN_RESET);
12             break;
13          case 2:
14             display7SEG(led_buffer[2]);
15             HAL_GPIO_WritePin(EN2_GPIO_Port, EN2_Pin, GPIO_PIN_RESET);
16             break;
17          case 3:
18             display7SEG(led_buffer[3]);
19             HAL_GPIO_WritePin(EN3_GPIO_Port, EN3_Pin, GPIO_PIN_RESET);
20             break;
21          default:
22             break;
23      }
```

```
1  const uint8_t segment_code[10] = {
2      0xC0, // 0
3      0xF9, // 1
4      0xA4, // 2
```

```
5  0xB0, // 3
6  0x99, // 4
7  0x92, // 5
8  0x82, // 6
9  0xF8, // 7
10 0x80, // 8
11 0x90  // 9
12 };
13
14 void display7SEG(int num) {
15     if (num < 0 || num > 9) return;
16
17     uint8_t code = segment_code[num];
18
19     HAL_GPIO_WritePin(SEG0_GPIO_Port, SEG0_Pin, (code & 0x01) ?
20     GPIO_PIN_SET : GPIO_PIN_RESET);
21     HAL_GPIO_WritePin(SEG1_GPIO_Port, SEG1_Pin, ((code >> 1) & 0x01) ?
22     GPIO_PIN_SET : GPIO_PIN_RESET);
23     HAL_GPIO_WritePin(SEG2_GPIO_Port, SEG2_Pin, ((code >> 2) & 0x01) ?
24     GPIO_PIN_SET : GPIO_PIN_RESET);
25     HAL_GPIO_WritePin(SEG3_GPIO_Port, SEG3_Pin, ((code >> 3) & 0x01) ?
26     GPIO_PIN_SET : GPIO_PIN_RESET);
27     HAL_GPIO_WritePin(SEG4_GPIO_Port, SEG4_Pin, ((code >> 4) & 0x01) ?
28     GPIO_PIN_SET : GPIO_PIN_RESET);
29     HAL_GPIO_WritePin(SEG5_GPIO_Port, SEG5_Pin, ((code >> 5) & 0x01) ?
30     GPIO_PIN_SET : GPIO_PIN_RESET);
31     HAL_GPIO_WritePin(SEG6_GPIO_Port, SEG6_Pin, ((code >> 6) & 0x01) ?
32     GPIO_PIN_SET : GPIO_PIN_RESET);
33 }
```

9 Exercise 9

This is an extra works for this lab. A LED Matrix is added to the system. A reference design is shown in figure bellow:

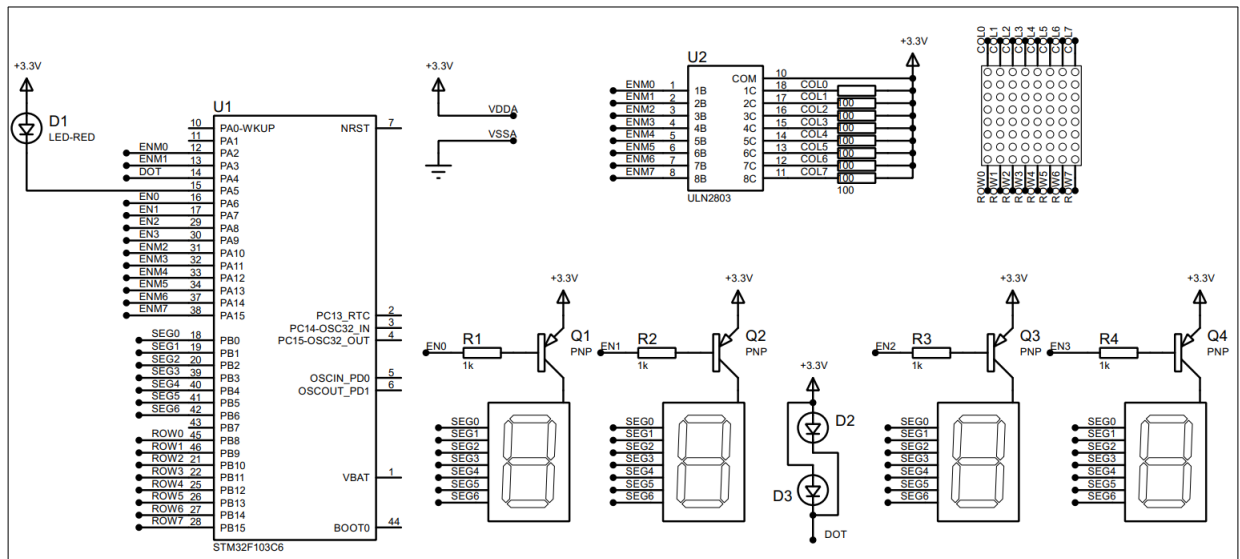


Figure 9.1: LED matrix is added to the simulation

In this schematic, two new components are added, including the **MATRIX-8X8-RED** and **ULN2803**, which is an NPN transistor array to enable the power supply for a column of the LED matrix. Students can change the enable signal (from ENM0 to ENM7) if needed. Finally, the data signal (from ROW0 to ROW7) is connected to PB8 to PB15.

Report 1: Present the schematic of your system by capturing the screen in Proteus.

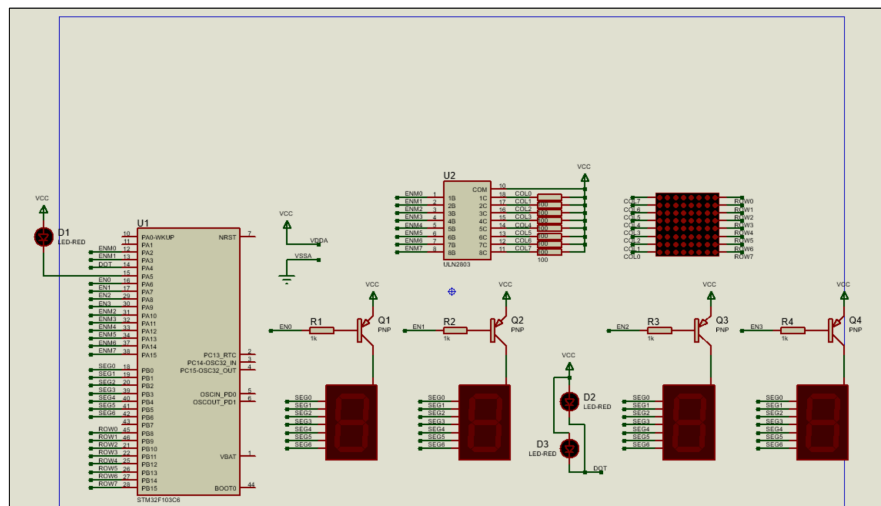


Figure 9.2: Proteus Schematic for Exercise 9. The source code and project files can be accessed at: [GitHub Repository](#)

Report 2: Implement the function, `updateLEDMatrix(int index)`, which is similarly to 4 seven led segments.



```
1 void updateLEDMatrix(int index) {
2     HAL_GPIO_WritePin(GPIOA, ENM0_Pin|ENM1_Pin|ENM2_Pin|
3         ENM3_Pin| ENM4_Pin|ENM5_Pin|
4         ENM6_Pin|ENM7_Pin, GPIO_PIN_RESET);
5
6     uint8_t row_data = matrix_buffer[index];
7
8     GPIOB->ODR = (GPIOB->ODR & 0x00FF) | ((uint32_t)row_data << 8);
9
10    switch (index) {
11        case 0:
12            HAL_GPIO_WritePin(ENM0_GPIO_Port, ENM0_Pin, GPIO_PIN_SET);
13            break;
14        case 1:
15            HAL_GPIO_WritePin(ENM1_GPIO_Port, ENM1_Pin, GPIO_PIN_SET);
16            break;
17        case 2:
18            HAL_GPIO_WritePin(ENM2_GPIO_Port, ENM2_Pin, GPIO_PIN_SET);
19            break;
20        case 3:
21            HAL_GPIO_WritePin(ENM3_GPIO_Port, ENM3_Pin, GPIO_PIN_SET);
22            break;
23        case 4:
24            HAL_GPIO_WritePin(ENM4_GPIO_Port, ENM4_Pin, GPIO_PIN_SET);
25            break;
26        case 5:
27            HAL_GPIO_WritePin(ENM5_GPIO_Port, ENM5_Pin, GPIO_PIN_SET);
28            break;
29        case 6:
30            HAL_GPIO_WritePin(ENM6_GPIO_Port, ENM6_Pin, GPIO_PIN_SET);
31            break;
32        case 7:
33            HAL_GPIO_WritePin(ENM7_GPIO_Port, ENM7_Pin, GPIO_PIN_SET);
34            break;
35        default:
```



```
36     break;
37 }
38 }
```

10 Exercise 10

Create an animation on LED matrix, for example, the character is shifted to the left.

Report 1: Briefly describe your solution and present your source code in the report.

To create a scrolling animation on the LED matrix where the character shifts from right to left, a multi-tasking approach utilizing software timers was implemented. The solution involves two independent tasks:

1. **High-Speed Display Scanning (timer3):** This timer runs continuously at a very high frequency (every 10ms, configured via `setTimer3(1)`). Its sole purpose is to rapidly iterate through the `matrix_buffer` and push the data to the LED matrix hardware to create a stable, flicker-free image
2. **Data Shifting Logic (timer4):** A new software timer (`timer4`) was introduced specifically to handle the animation timing. It is configured to trigger every 250ms (`setTimer4(25)`). When triggered, it performs a circular left shift on the `matrix_buffer` array. The first element (leftmost column) is temporarily saved, all subsequent elements are shifted one position to the left, and the saved element is moved to the last position (rightmost column).

Because the display scanning (`timer3`) operates independently and much faster than the data shifting (`timer4`), any modification made to the buffer by `timer4` is instantly reflected on the display, resulting in a smooth scrolling effect without blocking the main program loop or the 7-segment clock display.

Source Code:

```
1  setTimer3(1); // High-speed matrix scanning (10ms)
2  setTimer4(25); // Animation speed: shift every 250ms
3
4  // 2. Main Loop Execution
5  while (1)
6  {
7      if (timer1_flag == 1) {
```

```
8      setTimer1(25);
9      update7SEG(index_led);
10
11     index_led++;
12     if (index_led >= MAX_LED) {
13         index_led = 0;
14     }
15 }
16
17 if (timer2_flag == 1) {
18     setTimer2(100);
19     HAL_GPIO_TogglePin(DOT_GPIO_Port, DOT_Pin);
20
21     // logic counting time
22     second++;
23     if (second >= 60) {
24         second = 0;
25         minute++;
26     }
27     if (minute >= 60) {
28         minute = 0;
29         hour++;
30     }
31     if (hour >= 24) {
32         hour = 0;
33     }
34
35     updateClockBuffer();
36 }
37
38 // Task 3: Continuous matrix scanning (Runs constantly)
39 if (timer3_flag == 1) {
40     setTimer3(1);
41     updateLEDMatrix(index_led_matrix);
42     index_led_matrix++;
```

```
43     if (index_led_matrix >= MAX_LED_MATRIX) {  
44         index_led_matrix = 0;  
45     }  
46 }  
47  
48 // Task 4: Circular left shift for scrolling animation (Runs  
every 250ms)  
49 if (timer4_flag == 1) {  
50     setTimer4(25); // Reload 250ms  
51  
52     // Save the value of the leftmost column  
53     uint8_t temp = matrix_buffer[0];  
54  
55     // Shift all column data one position to the left  
56     for (int i = 0; i < 7; i++) {  
57         matrix_buffer[i] = matrix_buffer[i + 1];  
58     }  
59  
60     // Move the originally leftmost column to the rightmost  
position  
61     matrix_buffer[7] = temp;  
62 }  
63 }
```



References

- [1] Paul Adrien Maurice Dirac. *The Principles of Quantum Mechanics*. International series of monographs on physics. Clarendon Press, 1981.
- [2] Albert Einstein. Zur Elektrodynamik bewegter Körper. (German) [On the electrodynamics of moving bodies]. *Annalen der Physik*, 322(10):891–921, 1905.
- [3] Michel Goossens, Frank Mittelbach, and Alexander Samarin. *The L^AT_EX Companion*. Addison-Wesley, Reading, Massachusetts, 1993.
- [4] George D. Greenwade. The Comprehensive Tex Archive Network (CTAN). *TUGBoat*, 14(3):342–351, 1993.
- [5] Gareth Halfacree. The official bbc micro:bit user guide.
- [6] Donald Knuth. Knuth: Computers and typesetting.
- [7] Donald E. Knuth. *The Art of Computer Programming*. Four volumes. Addison-Wesley, 1968. Seven volumes planned.
- [8] Donald E. Knuth. *Fundamental Algorithms*, chapter 1.2. Addison-Wesley, 1973.